Information Security Policy

Company	Rushing Technologies
Product	ZenFinance (iOS application and API)
Policy owner	Nicholas Rushing, Founder
Security contact	support@rushingtechnologies.com (monitored)
Version	2.0
Adopted	2026-07-14
Next scheduled review	2027-01-14

1. Purpose and Scope

This policy establishes the requirements for protecting the confidentiality, integrity, and availability of information handled by Rushing Technologies in building and operating ZenFinance, a personal-finance application consisting of an iOS client, a Node.js API, and supporting infrastructure.

It applies to all systems, data, code, vendor relationships, and personnel of Rushing Technologies. Rushing Technologies is currently a single-member company; where a control below presumes multiple people (e.g., onboarding, separation of duties), the requirement is written so that it takes effect when the first additional person is granted access, and the compensating control for the single-member case is stated.

Requirement language follows RFC 2119: **MUST** denotes a mandatory control; **SHOULD** denotes an expected control that may be deferred with a documented exception under §4.

Supporting documents referenced throughout: `SECURITY_AUDIT.md` (assessment results and remediation log), `FAILURE_DRILLS.md` (failure-mode drills and validating tests), `APP_STORE_PRIVACY.md` (data inventory and processor disclosures), `DEPLOY.md` (production deployment and configuration reference).

2. Definitions

- **Restricted data** — data whose disclosure enables direct account or financial compromise: bank provider access tokens, password hashes, signing and encryption keys, API secrets, session-signing material.
- **Confidential data** — user personal and financial information: transactions, balances, account metadata, email addresses, goals, coaching artifacts, support tickets.

- **Internal data** — non-public operational material that is not tied to an identifiable user: aggregate metrics, source code, runbooks.
- **Public data** — material intended for publication (marketing site, aggregate launch statistics above the minimum sample size).
- **Processor** — a third-party service that receives company or user data in the course of providing its function.
- **Incident** — an event that has compromised, or presents a credible risk of compromising, the confidentiality, integrity, or availability of Restricted or Confidential data or of production systems.

3. Roles and Responsibilities

The **Founder** is the accountable owner of the information security program and performs the roles of security officer, incident commander, and risk owner. Responsibilities include: maintaining this policy; performing and recording risk assessments; approving and auditing access; selecting and reviewing processors; leading incident response; and ensuring remediation of findings.

If personnel or contractors are engaged, each person **MUST** be assigned access under §7, acknowledge this policy in writing before receiving access, and report suspected incidents to the security contact immediately.

4. Policy Governance, Review, and Exceptions

- This policy **MUST** be reviewed at least every six months and upon any material change to architecture, data categories, or processors. Reviews and revisions are recorded in §22.
- Exceptions to any **MUST** requirement **MUST** be documented in this file with scope, rationale, compensating control, and an expiry or re-review date. Undocumented deviations are treated as findings under §5.
- This policy is version-controlled in the product repository; changes flow through the same pull-request and review pipeline as code (§11).

5. Risk Management

Risk is assessed on a defined cadence and at defined trigger points, and each assessment's material findings **MUST** be remediated or accepted in writing.

- **Continuous (every pull request):** automated CI runs typechecking, the full API test suite, and a heuristic security scan; release branches additionally run an LLM-triaged scan in which CONFIRMED findings at or above the configured severity threshold fail the build (`.github/scripts/gate_security_scan.py`).
- **Monthly:** dependency vulnerability review via `npm audit --audit-level=high` (gate) with moderate advisories in non-production tooling tracked and re-checked in `SECURITY_AUDIT.md`.
- **Per release:** the release checklist in `SECURITY_AUDIT.md` (typecheck, API test suite against a dedicated test database, production build, dependency audit, native dependency verification).
- **Per architectural change:** any new data category, processor, or externally reachable surface **MUST** trigger an update to the data inventory (`APP_STORE_PRIVACY.md`), this policy, and, where user-visible, the public

privacy disclosure.

- **Scenario drills:** failure modes with security or data-integrity impact (provider webhook outage, item reauthentication, LLM provider failure, billing webhook failure, account-deletion failure) are documented in `FAILURE_DRILLS.md` together with expected behavior and the automated tests that validate each.

Accepted risks, applied remediations, and assessment results are recorded in `SECURITY_AUDIT.md`, which serves as the risk register and remediation log.

6. Data Classification and Handling

All data handled by ZenFinance **MUST** be treated according to its classification in §2.

- **Restricted** data **MUST NOT** leave the server environment, appear in logs or telemetry, or be sent to any LLM or analytics processor. Bank provider access tokens are additionally encrypted at the application layer before storage (§8) so that database access alone does not disclose them.
- **Confidential** data **MUST** be transmitted only over TLS, stored only in the production database or the processors enumerated in §17, and minimized before any LLM processing (compact, redacted summaries; never raw credentials or tokens).
- Bank connectivity is **read-only**. Card and payment details are never collected or stored; subscription state is handled by the App Store and RevenueCat, with only entitlement/product status retained.
- Financial and session API responses **MUST** carry `Cache-Control: no-store`.
- Production data **MUST NOT** be copied to development environments. Local development and CI use mock providers and dedicated test databases.

7. Access Control and Identity Management

- Access to production infrastructure (Railway), DNS and static hosting (Cloudflare), source control (GitHub), and processor dashboards (Plaid, RevenueCat, Sentry, Anthropic, Resend, Apple/Expo) is limited to the Founder. Multi-factor authentication **MUST** be enabled on every such account where the provider supports it.
- Access follows least privilege. If any additional person is engaged, they **MUST** receive a scoped account (never shared credentials), and their access **MUST** be revoked within one business day of engagement ending. With a single member, the compensating control is that no credential is shared and every provider login is individually attributable.
- **API authentication:** every user route requires a signed JWT; passwords are hashed with bcrypt; the admin console is gated by a separate high-entropy secret (minimum 32 characters, enforced at §9's fail-closed boot validation).
- **Abuse resistance:** per-user and per-route rate limits are enforced on sensitive endpoints, including authentication, link-token issuance, and public-token exchange.
- Sessions on device are stored in the iOS Keychain via SecureStore with device-bound accessibility (`WHEN_UNLOCKED_THIS_DEVICE_ONLY`).

8. Cryptography and Key Management

- **In transit:** TLS **MUST** be used for all client–API and API–processor traffic. The iOS app enforces App Transport Security with no arbitrary-loads exceptions.
- **At rest:** the production Postgres database is hosted on Railway with provider-managed encryption at rest. Bank provider access tokens are additionally encrypted at the application layer with AES-256-GCM under a dedicated 256-bit key (`TOKEN_ENC_KEY`) before storage.
- **Key generation:** secrets and keys **MUST** be generated with a cryptographically secure random source at 256-bit strength (e.g., `openssl rand -hex 32`).
- **Key storage:** keys and secrets live exclusively in platform environment configuration (Railway). They **MUST NOT** be committed to the repository; the repository contains only a placeholder `.env.example` .
- **Key rotation:** compromised or suspected-compromised keys **MUST** be rotated immediately as part of incident containment (§15). Rotating the token-encryption key orphans stored provider tokens by design; affected users are prompted to relink, which is the documented recovery path.
- **Webhook authenticity:** inbound webhooks **MUST** be cryptographically verified before processing. Plaid webhooks are verified as ES256 JWTs with key-ID resolution against Plaid's JWKS, an issued-at freshness window of 300 seconds, and a SHA-256 hash comparison of the raw request body. RevenueCat webhooks require both the shared-secret Authorization header and a valid HMAC signature.

9. Platform, Network, and Configuration Security

- The API **MUST** refuse to start when required secrets are missing or below minimum strength (fail-closed environment validation); it never falls back to development defaults in production.
- HTTP hardening headers are applied globally (Helmet). CORS is restricted to the first-party web origins in production. The API serves only `/api/*` ; all other paths return a uniform 404.
- Request bodies are limited to 128 KB and schema-validated (Zod) before handlers run. Central error handling returns uniform error shapes; stack traces are never sent to clients.
- Static sites (marketing, admin) are deployed as isolated Cloudflare Workers, separate from the API origin.
- Development workstations used to access production or Restricted data **MUST** use full-disk encryption, OS auto-updates, and an auto-locking screen, and **MUST NOT** store production secrets outside the provider dashboards or a password manager.

10. Secure Software Development Lifecycle

- All changes reach production through pull requests on GitHub. Every pull request runs automated review, typechecking, the API test suite (covering webhook-signature rejection, authorization, rate limiting, premium gating, and deletion paths, among others), and the security scan described in §5.
- Release branches are additionally gated by the triaged security scan; builds fail on confirmed findings at or above the configured severity.

- Direct pushes to production infrastructure are not part of the deployment path; Railway deploys from the repository.
- Dependencies are pinned via lockfile. New dependencies **SHOULD** be reviewed for maintenance status and known advisories before adoption; defense-in-depth overrides are applied for known-risky transitive paths.
- Telemetry hygiene is enforced in code: Sentry events (server and iOS) pass through recursive scrubbing of token-, secret-, credential-, cookie-, authorization-, and email-like keys before send, with `sendDefaultPii` disabled; server console logs use a log-safe error summary that excludes outgoing request configuration and authorization headers (third-party providers' error response bodies may be logged for failure diagnosis).

11. Change Management

- Every production change is a version-controlled commit associated with a pull request and its CI results, providing a complete audit trail of what changed, when, and why.
- Database schema changes are applied through versioned migrations (drizzle-kit), which are reviewed in the same pull-request flow.
- Rollback is performed by redeploying the previous known-good commit through the hosting platform.

12. Vulnerability and Patch Management

- **Detection:** monthly `npm audit` review (§5); automated CI scanning on every change; Sentry runtime error monitoring; processor security notices.
- **Remediation targets:** critical or actively exploited issues **MUST** be remediated or mitigated before the next deploy and in any case within 72 hours of confirmation; high within 14 days; moderate within 90 days or a documented acceptance in `SECURITY_AUDIT.md` where the affected path is not production-reachable.
- Applied remediations are recorded in `SECURITY_AUDIT.md`.

13. Logging, Monitoring, and Alerting

- Runtime errors and crashes on both the API and the iOS app are captured in Sentry with the PII scrubbing described in §10, tagged by route and release for triage.
- Provider integration failures are logged with log-safe summaries that include the provider's error body (never our credentials) so that root cause is diagnosable without exposing Restricted data.
- Security-relevant state transitions — item reauthentication requirements, disconnections, account deletions — are recorded; account deletion writes a non-PII audit event capturing item count, revocation-failure count, and completion time.
- Logs and telemetry are retained per the operating platform's and Sentry's configured retention and are not exported to additional systems.

14. Incident Response

Severity levels.

- **SEV-1:** confirmed unauthorized access to Restricted or Confidential data, or compromise of production credentials or provider tokens.
- **SEV-2:** an exploitable vulnerability exposing Restricted or Confidential data, or an authentication/authorization bypass, without evidence of exploitation.
- **SEV-3:** a control failure without data exposure (e.g., a gate skipped, a webhook verification misconfiguration caught internally).

Response procedure. Upon detection (telemetry, processor notice, user report to the security contact), the Founder **MUST**:

1. **Triage** severity and scope from telemetry, logs, and provider dashboards; open an incident record (timestamped notes retained with the `SECURITY_AUDIT.md` log).
2. **Contain** — rotate affected credentials and keys, revoke provider items, disable the affected surface, or roll back the deployment, as applicable.
3. **Eradicate and recover** — remediate root cause through the standard change pipeline; verify with the relevant tests or drills.
4. **Notify** — for SEV-1: affected users without undue delay and within any timeline required by applicable breach-notification law; Plaid and other affected processors per their incident-reporting obligations; for SEV-2: processors where their data or integration is implicated.
5. **Post-incident** — record the event, root cause, and remediation in `SECURITY_AUDIT.md`; update this policy, drills, or tests where the incident revealed a gap.

Targets: triage within 24 hours of detection; SEV-1 containment within 24 hours of confirmation.

15. Business Continuity and Disaster Recovery

- The API, database, and queue run on Railway managed services; static sites run on Cloudflare Workers. Platform-managed Postgres backups **MUST** be enabled in production (a required deployment-checklist item in `DEPLOY.md`), and the backup retention window **MUST** be reflected in the public privacy policy because it bounds deletion propagation.
- Recovery from infrastructure loss is by redeploying the version-controlled repository to the platform and restoring the database from the most recent platform backup. Secrets are re-entered from the password manager per §8.
- Degraded-mode behavior for dependency outages (Plaid webhooks, LLM provider, billing webhooks) is defined and tested in `FAILURE_DRILLS.md`; no user-facing request path hard-depends on an LLM call except premium chat, and brief generation falls back to a deterministic template.

16. Third-Party and Vendor Risk Management

- Processors are limited to those enumerated in `APP_STORE_PRIVACY.md`: Plaid (read-only bank connectivity), RevenueCat (subscription entitlements), Anthropic (enrichment and coaching from redacted summaries), Sentry (scrubbed diagnostics), Expo/APNs (push delivery), and Resend (support email) — plus Railway, Cloudflare, GitHub, and Apple as infrastructure.
- Each processor receives the minimum data necessary for its function (mapping maintained in `APP_STORE_PRIVACY.md`).
- Adding or replacing a processor **MUST** be treated as an architectural change under §5: data-inventory update, policy update, and privacy disclosure update before launch.
- Processor selection **SHOULD** prefer vendors with published security programs and independent attestations (the current set — Plaid, Stripe-tier billing infrastructure via RevenueCat, Sentry, Cloudflare, GitHub, Apple — all publish SOC 2 and/or equivalent programs).

17. Data Retention, Deletion, and User Rights

Users can at any time, in-app or via the API:

- **Disconnect** a linked institution — provider access is revoked and the item's accounts and transactions are hard-deleted (foreign-key cascade).
- **Export** their data (`GET /api/me/export`).
- **Delete** their account (`DELETE /api/me`) — provider items are revoked (revocation failure never blocks the user's deletion right and is retried via a pending-revocation process), all rows cascade-delete, and a non-PII deletion audit event is written.

Account data is retained only while the account exists. Residual copies in platform backups age out with the backup retention window disclosed in the privacy policy. Telemetry retention follows §13.

18. Personnel Security

Currently not applicable beyond the Founder. Before any employee or contractor is granted access to systems or Confidential data, they **MUST**: acknowledge this policy in writing; receive least-privilege, individually attributable access under §7; and use a workstation meeting §9. Security responsibilities end-of-engagement (access revocation, device sanitization) **MUST** be completed within one business day.

19. Acceptable Use

Company systems and data **MUST** be used only for operating and improving ZenFinance. Production Confidential data **MUST NOT** be accessed except for support (at a user's request), incident response, or engineering work that cannot be performed with test data — and never copied to personal or development systems.

20. Compliance

This program is designed to satisfy the data-safeguard expectations of the processors ZenFinance integrates with (including Plaid's information-security requirements for production access), Apple's App Store privacy requirements, and applicable U.S. state breach-notification laws. Rushing Technologies does not currently claim a formal certification (e.g., SOC 2, ISO 27001); §21 records the assurance activities actually performed.

21. Independent Assurance

- **Internal assessments:** performed and documented in `SECURITY_AUDIT.md` (most recent: July 2026), supplemented by automated scanning on every change (§5).
- **Automated review:** every pull request receives automated code review in addition to CI gates.
- **Independent penetration testing:** not yet performed. A third-party penetration test **SHOULD** be commissioned as the user base and team grow, and in any case before handling data categories beyond the current inventory; this section and §22 will be updated when it occurs.

22. Document Control

Version	Date	Change
1.0	2026-07-14	Initial adoption: policy rollup of existing practices.
1.1	2026-07-14	Scoped the log/telemetry scrubbing claim to match code guarantees (review finding).
2.0	2026-07-14	Expanded to full program structure: classifications, governance and exceptions, key management, SDLC, change management, vulnerability SLAs, incident severity levels and procedure, continuity/recovery, personnel and acceptable use, compliance posture.